Alex Goessmann

The Tensor Network approach to Neuro-Symbolic AI

February 27, 2026

Springer Nature

Contents

Notation ix
1.1 Tensors ix
1.2 Tensor networks and contractions x
1.3 Generic contractions xii
1.4 Normalizations xiii
1.5 Function encoding and Computation-Activation Networks xiii

Part I Tensor Foundations of Intelligence

2 Probability Theory 3

3 Propositional Logics 5

4 First-Order Logics 7

5 Circuits and Automata 9
5.1 The neural paradigm 9
5.1.1 Function decomposition 9
5.1.2 Function evaluation by message passing 13

6 Causality 17
6.1 Rung 1: Association 17
6.2 Rung 2: Intervention 18
6.2.1 Do calculus 18
6.2.2 Intervention Queries 19
6.2.3 Simplifications 19
6.3 Rung 3: Counterfactuals 21
6.3.1 Response Variables 22
6.3.2 Counterfactual Twinned Network 23

Part II Computation-Activation Networks

7 Elementary Computation-Activation Networks 27

8 Mean Polytopes and Maximum Entropy Distributions 29

9 Variational Inference 31

10 Inductive Reasoning 33

11 Probabilistic Guarantees 35

12 Quantum Inference 37

13 Probabilistic Relational Models 39

Part III Contraction Calculus

Glossary 43

Solutions 45

Acronyms

TT	Tensor-Train format
CP	Canonical Polyadic format
CAN	Computation-Activation Network
HLN	Hybrid Logic Network, Computation-Activation Network in case of binary variables

Chapter 1
Notation

Abstract We introduce the notation used in this book.

In this section, we introduce the hypergraph-based tensor network formalism and define the most general tensor network architecture of CompActNets based on this formalism.

1.1 Tensors

Tensors are multiway arrays and a generalization of vectors and matrices to higher orders. We first provide a formal definition as real maps from index sets enumerating the coordinates of vectors, matrices, and higher-order tensors. To ease the notation, we abbreviate sets as $[d] = \{0, \ldots, d-1\}$, tuples of state indices by $x_{[d]} = (x_0, \ldots, x_{d-1})$ and tuples of variables by $X_{[d]} = (X_0, \ldots, X_{d-1})$.

Definition 1.1 (Tensor) For $k \in [d]$, let $m_k \in \mathbb{N}$ and let X_k be categorical variables taking values in $[m_k]$. A tensor $\tau\left[X_0, \ldots, X_{d-1}\right]$ of order d and leg dimensions $m_0, \ldots, m_{d-1}$ is defined through its coordinates

$$\tau\left[X_{[d]} = x_{[d]}\right] = \tau\left[X_0 = x_0, \ldots, X_{d-1} = x_{d-1}\right] \in \mathbb{R}$$

for index tuples

$$x_{[d]} = (x_0, \ldots, x_{d-1}) \in \bigtimes_{k \in [d]} [m_k] \,.$$

Tensors $\tau\left[X_{[d]}\right]$ are elements of the tensor space

$$\bigotimes_{k \in [d]} \mathbb{R}^{m_k} \,,$$

which is a linear space, enriched with the operations of coordinate-wise summation and scalar multiplication. We call a tensor $\tau\left[X_{[d]}\right]$ boolean, when all coordinates are in $\{0, 1\}$, and positive, when all coordinates are greater than 0.

We introduced tensors here in a non-canonical way based on categorical variables assigned to their axes. While this may look like syntactic sugar at this point, it allows us to define contractions without further specification of axes, based on comparisons of shared variables. We occasionally also allow for variables X taking values in infinite sets such as $\mathbb{R}$, in which case we denote the set of values to a variable by $\mathrm{val}\,(X)$.

Example 1.1 (Delta tensor) Given a tuple of variables $X_{[d]} = (X_0, \ldots, X_{d-1})$ with identical dimension m, where $d \geq 1$, the delta tensor is the element

$$\delta^{[d],m}\left[X_{[d]}\right] \in \bigotimes_{k\in[d]} \mathbb{R}^m$$

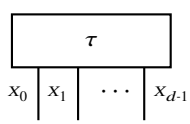
with coordinates

$$\delta^{[d],m}\left[X_{[d]} = x_{[d]}\right] = \begin{cases} 1 & \text{if} \quad x_0 = \ldots = x_{d-1} \\ 0 & \text{else} \end{cases} .$$

We depict this tensor by black dots, which sometimes appears as auxiliary elements in tensor network diagrams (see e.g. Figure 1.3). For $d = 1$, the delta tensor is the trivial vector, whose coordinates are constantly 1, which we denote by $\mathbb{I}\,[X]$.

1.2 Tensor networks and contractions

We use a standard visualization of tensors (dating back to [**?**]) by blocks with lines depicting the axes of the tensor. Additionally, we assign to each axis of the tensor the corresponding variable X_k:



We now associate categorical variables with nodes of a hypergraph and tensors with hyperedges, which are arbitrary subsets of nodes. Based on this association we continue with the definition of tensor networks.

Definition 1.2 (Tensor network) Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a hypergraph, let X_v for $v \in \mathcal{V}$ be categorical variables with dimensions $m_v \in \mathbb{N}$, and let

$$\tau^e\,[X_e] \in \bigotimes_{v\in e} \mathbb{R}^{m_v}$$

be tensors for $e \in \mathcal{E}$, where we denote by X_e the set of categorical variables X_v with $v \in e$. Then, we call the set

$$\tau^{\mathcal{G}}\left[X_{\mathcal{V}}\right] = \left\{\tau^{e}\left[X_{e}\right] \,:\, e \in \mathcal{E}\right\}$$

the tensor network of the decorated hypergraph $\mathcal{G}$. The set of tensor networks on $\mathcal{G}$ such that all tensors have non-negative coordinates is denoted by $\mathcal{T}^{\mathcal{G}}$.

As examples we now present the CP and the TT formats in our hypergraph notation.

Example 1.2 (The CP *format)* The Candecomp-Parafac (CP) tensor format (see [**?**]) corresponds in our notation to a hypergraph (see Figure 1.1) defined by

- nodes $X_{[d]}$ and a single hidden variable I, decorated by dimensions $m_{[d]}$ and the CP-rank n, respectively
- edges $\left\{e_k = \{X_k, I\} \,:\, k \in [d]\right\}$ each decorated by a matrix $\tau^{e_k}\left[X_k, I\right] \in \mathbb{R}^{m_k \times n}$.

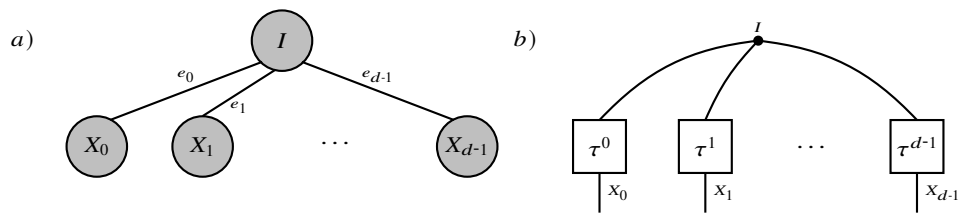


Fig. 1.1 Hypergraph to a CP format (see Example 1.2). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

Example 1.3 (The TT *format)* The Tensor-Train (TT) format (see [**?**]) corresponds in our notation to a hypergraph (see Figure 1.2) defined by

- nodes $X_{[d]}$ and hidden variables $I_{[d-1]}$, each decorated by a dimension $m_{[d]}$ and $n_{[d-1]}$,
- edges

  $$\left\{e_0 = \{X_0, I_0\}\right\} \cup \left\{e_k = \{I_{k-1}, X_k, I_k\} \,:\, k \in \{1, \ldots, d-2\}\right\} \cup \left\{e_{d-1} = \{I_{d-2}, X_{d-1}\}\right\}$$

  each decorated by a tensor of order 3 (respectively 2 for $k \in \{0, d-1\}$).

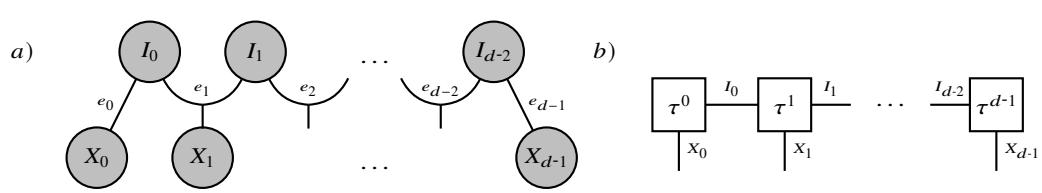


Fig. 1.2 Hypergraph to a TT format (see Example 1.3). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

1.3 Generic contractions

Let us now exploit our graphical approach to tensor networks in the definition of contractions.

Definition 1.3 Let $\tau^{\mathcal{G}}$ be a tensor network on a decorated hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. For any subset $\mathcal{U} \subset \mathcal{V}$ we define the contraction of $\tau^{\mathcal{G}}$ with open variables $X_{\mathcal{U}}$ to be the tensor (for an example see Figure 1.3)

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]} \in \bigotimes_{v \in \mathcal{U}} \mathbb{R}^{m_v}$$

with coordinates at indices $x_{\mathcal{U}} \in \bigtimes_{v \in \mathcal{U}} [m_v]$ by

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \sum_{x_{\mathcal{V} \setminus \mathcal{U}} \in \bigtimes_{v \in \mathcal{V} \setminus \mathcal{U}} [m_v]} \left(\prod_{e \in \mathcal{E}} \tau^e \left[X_e = x_e \right] \right).$$

When an open variable X does not appear in any tensor in a contraction, we define the contraction as a tensor product with the trivial tensor $\mathbb{I}[X]$ (see Example 1.1). To ease notation, we often omit the set notation by brackets $\{\cdot\}$.

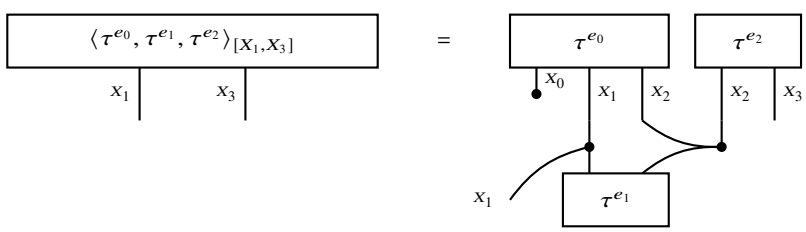


Fig. 1.3 Graphical depiction of a tensor network contraction with the open variables X_1, X_3. Open variables are depicted by those without a dot at the end of the line.

Example 1.4 (Tensor product) The simplest contraction is the tensor product, which maps a pair of two tensors with distinct variables onto a third tensor and has an interpretation by coordinate-wise products. Such a contraction corresponds with a tensor network of two tensors with disjoint variables. Let there be two tensors

$$\tau \left[X_{[d]} \right] \in \bigotimes_{k \in [d]} \mathbb{R}^{m_k} \quad \text{and} \quad \tilde{\tau} \left[Y_{[p]} \right] \in \bigotimes_{\ell \in [p]} \mathbb{R}^{n_\ell}$$

with disjoint tuples of categorical variables assigned to their axes. Then their tensor product is the tensor

$$\left\langle \tau\left[X_{[d]}\right], \tilde{\tau}\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}, Y_{[p]}\right]} \in \left(\bigotimes_{k\in[d]} \mathbb{R}^{m_k}\right) \otimes \left(\bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}\right)$$

with coordinates to tuples of $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ and $y_{[p]} \in \bigtimes_{\ell\in[p]}[n_\ell]$ as

$$\begin{aligned}&\left\langle \tau\left[X_{[d]}\right], \tilde{\tau}\left[Y_{[p]}\right]\right\rangle_{\left[X_{[d]}=x_{[d]}, Y_{[p]}=y_{[p]}\right]}\\ &\quad := \tau\left[X_{[d]} = x_{[d]}\right] \cdot \tilde{\tau}\left[Y_{[p]} = y_{[p]}\right] .\end{aligned}$$

1.4 Normalizations

Based on generic contractions, we now introduce the normalization of tensors, which introduces certain contraints on tensors to be depicted by directed hyperedges.

Definition 1.4 The normalization of a tensor $\tau\left[X_{\mathcal{V}}\right]$ on incoming nodes $\mathcal{V}^{\text{in}} \subset \mathcal{V}$ and outgoing nodes $\mathcal{V}^{\text{out}} \subset \mathcal{V}\backslash\mathcal{V}^{\text{in}}$ is the tensor $\langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}^{\text{out}}}|X_{\mathcal{V}^{\text{in}}}\right]}$ defined for $x_{\mathcal{V}^{\text{in}}}$ as

$$\langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}^{\text{out}}}|X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]} = \begin{cases} \frac{\langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{out}}}, X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]}}{\langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]}} & \text{if} \quad \langle\tau\rangle_{\left[X_{\mathcal{V}^{\text{in}}}=x_{\mathcal{V}^{\text{in}}}\right]} \neq 0 \\ \frac{1}{\prod_{v\in\mathcal{V}^{\text{out}}} m_v}\mathbb{I}\left[X_{\mathcal{V}^{\text{out}}}\right] & \text{else} \end{cases} .$$

We say that $\tau\left[X_{\mathcal{V}}\right]$ is normalized with incoming nodes $\mathcal{V}^{\text{in}} \subset \mathcal{V}$, if

$$\tau\left[X_{\mathcal{V}}\right] = \langle\tau\left[X_{\mathcal{V}}\right]\rangle_{\left[X_{\mathcal{V}\backslash\mathcal{V}^{\text{in}}}|X_{\mathcal{V}^{\text{in}}}\right]} .$$

In our graphical tensor notation, we depict normalized tensors by directed hyperedges (a), which are decorated by directed tensors (b), for example when $X_{\mathcal{V}^{\text{in}}} = (X_2, X_3)$ and $X_{\mathcal{V}\backslash\mathcal{V}^{\text{in}}} = (X_0, X_1)$:

1.5 Function encoding and Computation-Activation Networks

Towards presenting the function encoding schemes, we define one-hot encodings mapping the states of variables to basis tensors.

Definition 1.5 (One-hot encoding) To any variable X taking values in $[m]$, the one-hot encoding of any state $x \in [m]$ is the vector with coordinates

$$\epsilon_x \left[X = \tilde{x}\right] := \begin{cases} 1 & \text{if} \quad x = \tilde{x} \\ 0 & \text{else}\,. \end{cases}$$

To any tuple $X_{[d]}$ of variables taking values in $\bigtimes_{k\in[d]}[m_k]$, the one-hot encoding of a state tuple $x_{[d]}$ is the tensor product

$$\epsilon_{x_{[d]}}\left[X_{[d]}\right] := \bigotimes_{k\in[d]} \epsilon_{x_k}\left[X_k\right]\,.$$

We now use one-hot encodings to encode functions between state sets.

Definition 1.6 (Basis encoding of maps between state sets) Let there be two sets of variables $X_{[d]}$ and $Y_{[p]}$, and let there be a map

$$q \;:\; \bigtimes_{k\in[d]} [m_k] \to \bigtimes_{\ell\in[p]} [n_\ell]$$

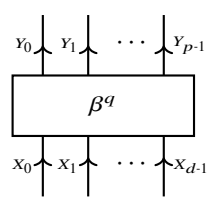
between their state sets. Then, the basis encoding of q is a tensor

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] \in \left(\bigotimes_{\ell\in[p]} \mathbb{R}^{n_\ell}\right) \otimes \left(\bigotimes_{k\in[d]} \mathbb{R}^{m_k}\right)$$

defined by

$$\beta^q\left[Y_{[p]}, X_{[d]}\right] = \sum_{x_{[d]}\in\bigtimes_{k\in[d]}[m_k]} \epsilon_{q\left(x_{[d]}\right)}\left[Y_{[p]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right]\,.$$

Basis encodings are normalized tensors and are thus depicted as decorations of directed edges in hypergraphs:



We further generalize basis encodings to arbitrary functions between finite sets by the use of bijective image enumeration maps. Given an arbitrary set $\mathcal{U}$, we say a map

$$I \;:\; \bigtimes_{k\in[d]} [m_k] \to \mathcal{U}$$

is an enumeration map of $\mathcal{U}$ by d variables X_k, taking values in m_k. Given a function $q : \mathcal{U}^{\text{in}} \to \mathcal{U}^{\text{out}}$ between arbitrary sets and enumerating maps I_{in} and I_{out} for both sets, we define the basis encoding of q as

$$\beta^q \left[Y_{[p]}, X_{[d]}\right] = \sum_{u \in \mathcal{U}^{\text{in}}} \epsilon_{I_{\text{out}}^{-1}(q(u))} \left[Y_{[p]}\right] \otimes \epsilon_{I_{\text{in}}^{-1}(u)} \left[X_{[d]}\right] ,$$

where $X_{[d]}, Y_{[p]}$ are variables taking values in $[\left|\mathcal{U}^{\text{in}}\right|]$ and $[\left|\mathcal{U}^{\text{out}}\right|]$. In Example 5.1 we present index enumeration maps for summations in m-adic integer representations. Based on these concepts, we define the most general tensor network architecture to be applied in the rest of this work.

Definition 1.7 (Computation-Activation Network (CompActNets)) Let there be a function $t : \bigtimes_{k \in [d]} [m_k] \to \mathbb{R}^p$ with basis encoding $\beta^t \left[Y_{[p]}, X_{[d]}\right]$, where $Y_{[p]}$ is a tuple of variables to an enumeration map of the image of t. Let there further be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with nodes $\mathcal{V}$ containing $[p]$. We define the by t computable and by $\mathcal{G}$ activated family of distributions by

$$\Lambda^{t,\mathcal{G}} = \left\{ \left\langle \beta^t \left[Y_{[p]}, X_{[d]}\right], \langle \xi \rangle_{\left[Y_{[p]}\right]} \right\rangle_{\left[X_{[d]} | \varnothing\right]} : \xi \left[Y_{\mathcal{V}}\right] \in \mathcal{T}^{\mathcal{G}} \right\} .$$

We refer to any member $\mathbb{P}\left[X_{[d]}\right] \in \Lambda^{t,\mathcal{G}}$ as a *Computation-Activation Network* (or shorter as a *CompActNet*). We call $\beta^t \left[Y_{[p]}, X_{[d]}\right]$ (and any decomposition of it) the *computation network* and $\xi \left[Y_{\mathcal{V}}\right]$ the *activation network*.

The elementary activated networks are representable by an elementary activation tensor with respect to the graph

$$\text{EL} = \left(\mathcal{V}, \{\{v\} : v \in \mathcal{V}\}\right)$$

and we denote such networks by $\Lambda^{t,\text{EL}}$. Any CompActNet is representable with respect to the maximal hypergraph

$$\text{MAX} = \left(\mathcal{V}, \{\mathcal{V}\}\right) .$$

We therefore have for any graph that $\Lambda^{t,\mathcal{G}} \subset \Lambda^{t,\text{MAX}}$.

Part I
Tensor Foundations of Intelligence

Many concepts of Artificial Intelligence can be captured by the formalism of tensors, tensor networks and their contractions. We dedicate this part to anchoring the formalism in the probabilisitc and logical foundations of artificial intelligence.

Chapter 2
Probability Theory

Abstract We present the main representation and the reasoning mechansims of probability theory, together with graphical models.

Topics:

- Chapter I.2 except for exponential families (later in book II.0)
- Sufficient statistics from sketches

Chapter 3
Propositional Logics

Abstract We present the representation and the reasoning mechansims of propositional logics.

Topics:

- Chapter I.4
- Chapter I.5

Chapter 4
First-Order Logics

Abstract To introduce the syntax and semantics of first-order logics, we in this chapter introduce the substitution tensor structure. This allows to capture structured representations of systems, beyond the factored representations introduced in the first two chapters.

Chapter 5
Circuits and Automata

Abstract We here present tensor network representation of the circuit model of computation.

Topics:

- Decomposition hypergraphs generalizing the syntactical hypergraphs in PL, to be called *circuits*
- Automata from the sketches

5.1 The neural paradigm

The neural paradigm of artificial intelligence exploits the decomposition of functions into neurons, which are aligned in a directed acyclic graph. We show in this section how functions decomposable into neurons can be represented by tensor networks. To this end, we formalize discrete neural models by decomposition graphs and formally prove the corresponding decomposition of their basis encodings.

5.1.1 Function decomposition

As a main principle of tensor decompositions, we now show that basis encodings of composition functions are contractions of the basis encodings of their components.

Lemma 5.1 *Let $f\left[X_{[d]}\right]$ be a composition of a p-ary connective function h and functions $f_\ell\left[X_{[d]}\right]$, where $\ell \in [p]$, that is for $x_{[d]} \in \bigtimes_{k\in[d]}[2]$ we have*

$$f(x_{[d]}) = h\left(f_0\left[x_{[d]}\right], \ldots, f_{p-1}\left[x_{[d]}\right]\right) .$$

Then, we have (see Figure 5.1)

$$\beta^f\left[Y_f, X_{[d]}\right] = \left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell}\left[Y_\ell, X_{[d]}\right] : \ell \in [p]\}\right\rangle_{\left[Y_f, X_{[d]}\right]} .$$

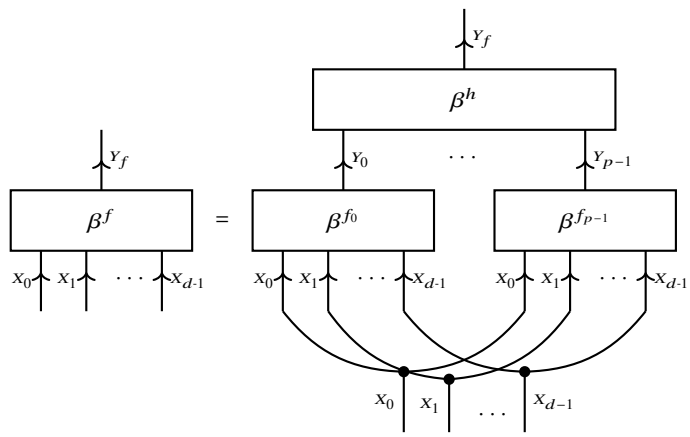


Fig. 5.1 Tensor network decomposition of the basis encoding of a function f, which is the composition of the functions $f_0, \ldots, f_{p-1}$ with a function h.

Proof. For any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ we have

$$\begin{aligned}
&\left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell}\left[Y_\ell, X_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f, X_{[d]}=x_{[d]}]} \\
&\quad = \left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\beta^{f_\ell}\left[Y_\ell, X_{[d]} = x_{[d]}\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f]} \\
&\quad = \left\langle \{\beta^h\left[Y_f, Y_{[p]}\right]\} \cup \{\epsilon_{f_\ell(x_{[d]})}\left[Y_\ell\right] \,:\, \ell \in [p]\}\right\rangle_{[Y_f]} \\
&\quad = \epsilon_{f(x_{[d]})}\left[Y_f\right] \\
&\quad = \beta^f\left[Y_f, X_{[d]} = x_{[d]}\right] .
\end{aligned}$$

Thus, the tensors on both sides of the equation coincide in all slices to $X_{[d]}$ and are thus equal. □

We now define a more generic decomposition of discrete functions.

Definition 5.1 A *decomposition hypergraph* is a directed acyclic hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ such that the following holds.

- Each node $v \in \mathcal{V}$ is decorated by a set $\mathcal{U}^v$ of finite cardinality m_v, a variable X_v, and an index interpretation function

$$I_v \,:\, [m_v] \to \mathcal{U}^v .$$

- Each directed hyperedge $(e^{\text{in}}, e^{\text{out}})$ has at least one outgoing node, that is $e^{\text{out}} \neq \emptyset$, and is decorated by an activation function

$$g^e \,:\, \bigtimes_{v\in e^{\text{in}}} \mathcal{U}^v \to \bigtimes_{v\in e^{\text{out}}} \mathcal{U}^v .$$

- Each node $v \in \mathcal{V}$ appears at most once as an outgoing node.
- The nodes not appearing as an outgoing node are enumerated by $v^{\text{in}}_{[d]}$. We abbreviate the corresponding variables by $X_{v^{\text{in}}_{[d]}} = X_{[d]}$.
- The nodes not appearing as an incoming node are enumerated by $v^{\text{out}}_{[p]}$. We abbreviate the variables by $X_{v^{\text{out}}_{[\ell]}} = Y_{[p]}$.

We assign for each $k \in [d]$ restriction functions

$$q_{v^{\text{in}}_k} : \bigtimes_{\tilde{k} \in [d]} \mathcal{U}^{v^{\text{in}}_{\tilde{k}}} \to \mathcal{U}^{v^{\text{in}}_k} \quad , \quad x_{[d]}|_k = x_k$$

to the nodes $v^{\text{in}}_{[d]}$ and recursively assign to each further node v a node function

$$q_v : \bigtimes_{k \in [d]} \mathcal{U}^{v^{\text{in}}_k} \to \mathcal{U}^v \quad , \quad q_v(x_{[d]}) = g^{e_v}\left(\bigtimes_{\tilde{v} \in e^{\text{in}}} q_{\tilde{v}}(x_{[d]})\right)|_v \, ,$$

where e_v is to each $v \in e^{\text{in}}$ the unique hyperedge with outgoing nodes $\{v\}$. We then call the function

$$q_{\mathcal{G}} : \bigtimes_{k \in [d]} \mathcal{U}^{v^{\text{in}}_k} \to \bigtimes_{\ell \in [p]} \mathcal{U}^{v^{\text{out}}_\ell} \quad , \quad q_{\mathcal{G}} = \bigtimes_{\ell \in [p]} q_{v^{\text{out}}_\ell}$$

the composition function to the decomposition hypergraph $\mathcal{G}$.

The neural paradigm in AI can be modeled by the existence of decomposition hypergraphs for functions on large sets. We now show how decomposition hypergraphs enable the sparse representation of composition functions by tensor networks.

Theorem 5.1 *For any decomposition hypergraph $\mathcal{G}$ with composition formula $q_{\mathcal{G}}$, we have*

$$\beta^{q_{\mathcal{G}}}\left[Y_{[p]}, X_{[d]}\right] = \left\langle \left\{ \beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \, : \, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} \, .$$

Proof. We show by induction over the number of edges in $\mathcal{G}$ that, for any $x_{[d]} \in \bigtimes_{k \in [d]} [m_k]$ and $v \in \mathcal{V}$, we have

$$\left\langle \left\{ \beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \, : \, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{\left[X_{\mathcal{V} \setminus v^{\text{in}}_{[d]}}, X_{[d]} = x_{[d]}\right]} = \bigotimes_{v \in \mathcal{V} \setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \, . \tag{5.1}$$

$|\mathcal{E}| = 1$: If there is a single edge $e = (e^{\text{in}}, e^{\text{out}})$ in $\mathcal{E}$, we have $X_{[d]} = X_{e^{\text{in}}}$ and $\mathcal{V} \setminus v^{\text{in}}_{[d]} = e^{\text{out}}$. In this case (5.1) holds since

$$\beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}} = x_{e^{\text{in}}}\right] = \bigotimes_{v \in \mathcal{V} \setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \, .$$

$(|\mathcal{E}| = n) \Rightarrow (|\mathcal{E}| = n+1)$: Let us now assume that (5.1) holds for all graphs with $|\mathcal{E}| \leq n$ and let $\mathcal{G}$ be a graph with $|\mathcal{E}| = n+1$. We find an edge $\tilde{e} = (e^{\text{in}}, e^{\text{out}})$ such that all nodes in e^{in} are only appearing as outgoing nodes in other edges.

$$
\begin{aligned}
&\left\langle \left\{ \beta^{g^e}\left[X_e\right] \,:\, e \in \mathcal{E} \right\} \right\rangle_{\left[X_{\mathcal{V}\setminus v^{\text{in}}_{[d]}}, X_{[d]} = x_{[d]}\right]} \\
&= \left\langle \beta^{\tilde{e}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right], \left\langle \left\{ \beta^{g^e}\left[X_e\right] \,:\, e \in \mathcal{E}\setminus\{\tilde{e}\} \right\} \right\rangle_{\left[X_{\mathcal{V}\setminus\{v^{\text{in}}_{[d]}\cup e^{\text{out}}\}}, X_{[d]} = x_{[d]}\right]} \right\rangle_{\left[X_{\mathcal{V}\setminus v^{\text{in}}_{[d]}}, X_{[d]} = x_{[d]}\right]} \\
&= \left\langle \beta^{\tilde{e}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right], \bigotimes_{v \in \mathcal{V}\setminus\{v^{\text{in}}_{[d]}\cup e^{\text{out}}\}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \right\rangle_{\left[X_{\mathcal{V}\setminus v^{\text{in}}_{[d]}}, X_{[d]} = x_{[d]}\right]} \\
&= \bigotimes_{v \in \mathcal{V}\setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .
\end{aligned}
$$

Here, in the second equation we used the induction hypothesis on the subgraph $(\mathcal{V}, \mathcal{E}\setminus\{\tilde{e}\})$ with $|\mathcal{E}\setminus\{\tilde{e}\}| = n$ and a generic contraction property of basis encodings in the third equation.

Thus, (5.1) holds always and we have for any $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ that

$$
\begin{aligned}
\left\langle \left\{ \beta^{g^e}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right] \,:\, e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{\left[Y_{[p]}, X_{[d]} = x_{[d]}\right]} &= \left\langle \bigotimes_{v \in \mathcal{V}\setminus v^{\text{in}}_{[d]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \right\rangle_{\left[Y_{[p]}, X_{[d]}\right]} \\
&= \bigotimes_{v \in v^{\text{out}}_{[p]}} \epsilon_{q_v(x_{[d]})}\left[X_v\right] \\
&= \epsilon_{q_{\mathcal{G}}}\left[Y_{[p]}\right] \\
&= \beta^{q_{\mathcal{G}}}\left[Y_{[p]}, X_{[d]} = x_{[d]}\right] .
\end{aligned}
$$

Keeping $X_{[d]}$ open, the claim is established. □

When neurons have tunable parameters, we can discretize those by sets $\mathcal{U}^k$ and understand them as additional input variables.

Example 5.1 (Sum of integers in m-adic representation) We develop a tensor network representation of integer summations on the set $[m^d] = \{0, \ldots, m^d - 1\}$, where $m, d \in \mathbb{N}$,

$$
+ \,:\, [m^d] \times [m^d] \to [m^{d+1}] \quad , \quad +(i, \tilde{i}) = i + \tilde{i} ,
$$

which have a m-adic representation of length d. We define an index interpretation map

$$
I \,:\, \bigtimes_{k\in[d]} [m] \to [m^d] \quad , \quad I(x_{[d]}) = \sum_{k\in[d]} x_k \cdot m^k ,
$$

which enables the parameterization of $[m^d]$ as the states of d categorical variables $X_{[d]}$ of dimension m. We analogously represent a second set $[m^d]$ by variables $\tilde{X}_{[d]}$ and the set $[m^{d+1}]$ of possible sums by $Y_{[d+1]}$. The basis encoding of the sum is then

$$\beta^+\left[Y_{[d+1]}, X_{[d]}, Y_{[d]}\right] = \sum_{x_{[d]},\tilde{x}_{[d]}} \epsilon_{I^{-1}(I(x_{[d]})+I(\tilde{x}_{[d]}))}\left[Y_{[d+1]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \otimes \epsilon_{\tilde{x}_{[d]}}\left[\tilde{X}_{[d]}\right] .$$

Note that the tensor space of β^+ is of dimension $m^{3\cdot d+1}$ increasing exponentially in d. Feasible representation of this tensor for large d therefore requires tensor network decompositions, which we now provide based on a decomposition hypergraph. The targeted function to be decomposed is the representation of the integer sum by

$$+^m : \left(\bigtimes_{k\in[d]}[m]\right)\times\left(\bigtimes_{k\in[d]}[m]\right)\to \bigtimes_{k\in[d+1]}[m] \quad , \quad +^m(x_{[d]},\tilde{x}_{[d]}) = I^{-1}(I(x_{[d]})+I(\tilde{x}_{[d]})) .$$

We build a decomposition hypergraph $\mathcal{G} = (\mathcal{V},\mathcal{E})$ (see Def. 5.1) consisting of $4\cdot d$ nodes (see Figure 5.2a) . The nodes carry the $(3\cdot d+1)$ variables $X_{[d]}, Y_{[d]}, Y_{[d+1]}$ of dimension m constructed above and $d-1$ auxiliary variables $Z_{[d-1]}$ of dimension 2 representing carry bits. The directed hyperedges of $\mathcal{G}$ are

$$\begin{aligned}\mathcal{E} = &\left\{(\{X_0,\tilde{X}_0\},\{Y_0,Z_0\})\right\} \cup \left\{(\{Z_{k-1},X_k,\tilde{X}_k\},\{Y_k,Z_k\}) \,:\, k\in\{1,\dots,d-2\}\right\}\\ &\cup \left\{(\{Z_{d-2},X_{d-1},\tilde{X}_{d-1}\},\{Y_{d-1},Y_d\})\right\}\end{aligned}$$

and are decorated by local summation functions

$$\tilde{+} \,:\, [2]\times[m]\times[m]\to[m]\times[2] \quad , \quad \tilde{+}(z,x,\tilde{x}) = \left((z+x+\tilde{x}) \bmod m, \left\lfloor\frac{z+x+\tilde{x}}{m}\right\rfloor\right) .$$

Since to the first hyperedge we do not have a carry bit, the decorating function is the restriction of the first argument to 0.

It is known that the composition of the local summations $\tilde{+}$ is the global summation $+^m$ of integers in m-adic representation. Thus, the composition function $q_{\mathcal{G}}$ is $+^m$. By Thm. 5.1 we have a decomposition of the basis encoding to $q_{\mathcal{G}}$ (see Figure 5.2b) as

$$\begin{aligned}\beta^{+^m}\left[Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}\right] = \Big\langle &\{\beta^{\tilde{+},0}\left[Y_0,Z_0,X_0,\tilde{X}_0\right]\}\cup\\ &\{\beta^{\tilde{+},k}\left[Y_k,Z_k,X_k,\tilde{X}_k,Z_{k-1}\right] \,:\, k\in\{1,\dots,d-2\}\}\cup\\ &\{\beta^{\tilde{+},d-2}\left[Y_{d-1},Y_d,X_{d-1},\tilde{X}_{d-1},Z_{d-2}\right]\}\Big\rangle_{\left[Y_{[d+1]},X_{[d]},\tilde{X}_{[d]}\right]} .\end{aligned}$$

5.1.2 Function evaluation by message passing

We are now concerned with an efficient inference algorithm based on tensor network contractions. To evaluate a function given as a tensor network decomposition of its

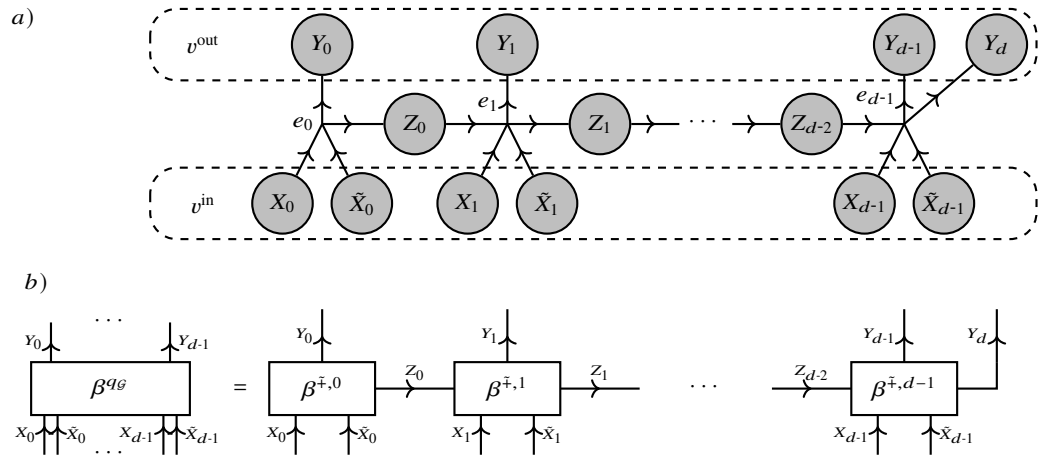


Fig. 5.2 Example of a decomposition hypergraph to the sum of integers (see Example 5.1). a) Hypergraph of directed edges e_k for $k \in [d]$, each decorated by an integer summation + preparing an index Y_k of the resulting sum. b) Corresponding tensor network decomposition of the basis encoded composition function, which is the sum of integers in m-adic representation.

basis encoding, the whole network has to be contracted. As this can be infeasible for large networks, a message passing algorithm based on local contractions can be applied, compare Algorithm **??** for a message passing algorithm for tensor networks on a on tree hypergraph.

Algorithm 1 Directed Belief Propagation

Require: Tensor network $\tau^{\mathcal{G}}$ on a directed hypergraph $\mathcal{G}$
Ensure: Messages $\{\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] : (e_0, e_1) \in \mathcal{E}^{\to}\}$

Prepare directed message directions

$$\mathcal{E}^{\to} = \left\{\left((e_0^{\text{in}}, e_0^{\text{out}}), (e_1^{\text{in}}, e_1^{\text{out}})\right) : e_0^{\text{in}} \cap (e_1^{\text{in}}, e_1^{\text{out}}) = \varnothing,\ e_1^{\text{out}} \cap (e_0^{\text{in}}, e_0^{\text{out}}) = \varnothing,\ e_0^{\text{out}} \cap e_1^{\text{in}} \neq \varnothing\right\}$$

Initialize a message queue $S = \{(e_2, e_0) : e_2 \text{ has empty incoming nodes}\}$
while S not empty **do**
Pop a (e_0, e_1) pair from S
Update the message

$$\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] = \left\langle \{\tau^{e_0}\left[X_{e_0}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] : (e_2, e_0) \in \mathcal{E}^{\to},\ e_2 \neq e_1\}\right\rangle_{\left[X_{e_0 \cap e_1}\right]}$$

Update S by all messages (e_1, e_3) which have not yet been sent, if all messages (e_2, e_1) have been sent.
end while
return Messages $\{\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] : (e_2, e_0) \in \mathcal{E}^{\to}\}$

We apply the Directed Belief Propagation, Algorithm Algorithm 1, on a decomposition hypergraph, where we add hyperedges to each leaf node and assign one-hot encodings of input states. We then show that the messages are the one-hot encodings to the evaluations of the node functions.

Theorem 5.2 *Let $\mathcal{G}$ be a decomposition graph and let us add hyperedges containing single input nodes, which are decorated by one-hot encodings. Then the messages computed in Algorithm 1 are characterized by*

$$\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] = \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .$$

Proof. We show the theorem inductively over the messages computed in Algorithm 1. The first message is sent from an input edge $\{[k]\}$ to an edge e of the decomposition graph and is by assumption the one-hot encoding of an input state $\epsilon_{x_k}\left[X_k\right]$.

We now assume that all previous messages satisfy the claimed equation at an arbitrary stage of the algorithm. The message computed in the while loop is then a contraction of one-hot encodings with basis encodings and

$$\begin{aligned}
\chi_{e_0 \to e_1}\left[X_{e_0 \cap e_1}\right] &= \left\langle \{\beta^{g^{e_0}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right]\} \cup \{\chi_{e_2 \to e_0}\left[X_{e_2 \cap e_0}\right] \,:\, (e_2, e_0) \in \mathcal{E}^{\to}\}\right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \left\langle \{\beta^{g^{e_0}}\left[X_{e^{\text{out}}}, X_{e^{\text{in}}}\right]\} \cup \{\epsilon_{q_v(x_{[d]})}\left[X_v\right] \,:\, v \in e^{\text{in}}\}\right\rangle_{\left[X_{e_0 \cap e_1}\right]} \\
&= \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})}\left[X_v\right] .
\end{aligned}$$

Thus, the new message is also the tensor product of the one-hot encodings of the evaluated node functions. By induction, the property is therefore true for all messages. □

We notice that we can interpret any directed acyclic hypergraph for which each node appears exactly once as an outgoing node and which is decorated by boolean and directed tensors $\tau^{\mathcal{G}}$. Edges with empty incoming sets are carrying one-hot encodings of input states and all further edges carry functions.

Example 5.2 (Continuation of Example 5.1) We now show how Algorithm 1 can be exploited to compute an efficient message passing algorithm for the digits of the m-adic sum. Given two numbers in m-adic representation by the tuples $x_{[d]}$ and $\tilde{x}_{[d]}$, we add the hyperedges with empty incoming nodes and single outgoing node

$$\bigcup_{k \in [d]} \left\{(\emptyset, \{X_k\}), (\emptyset, \{\tilde{X}_k\})\right\}$$

to the hypergraph of Example 5.1 and decorate them by the digit one-hot encodings $\epsilon_{x_k}\left[X_k\right]$ and $\epsilon_{\tilde{x}_k}\left[\tilde{X}_k\right]$ (see Figure 5.3). We then apply the Directed Belief Propagation Algorithm 1. The initial messages queue then consists of the messages from the digit encoding. As sketched in Figure 5.3, to each digit there are three messages (with the exception of the first being two), which can be scheduled in consecutive epochs $\chi^{(k,[3])}$. We then have by Thm. 5.2 for $k \in [d-1]$ that

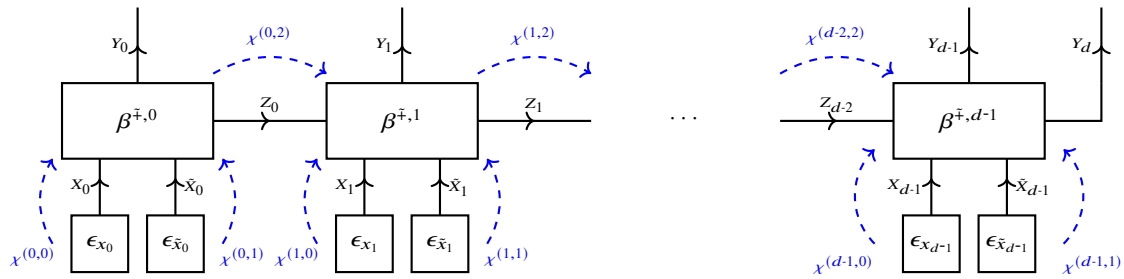


Fig. 5.3 Computation of the integer sum in m-adic representation by the Directed Belief Propagation Algorithm 1 (see Example 5.2). The summands are represented by one-hot encodings of the digits $x_{[d]}$ and $\tilde{x}_{[d]}$, from which the messages start. The k-th digit (for $k \in \{0, \ldots, d-1\}$) of the sum is computed based on the first messages of the epoch labeled by $\chi^{(k,[2])}$, The third message $\chi^{(k,2)}$ in each epoch communicates the carry bit to the next digit summation core. In the last message epoch the digit $d-1$ and d are computed based.

$$\left\langle \beta^{\mp,k}\left[Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}\right], \chi^{(k-1,2)}[Z_{k-1}], \chi^{(k,0)}[X_k], \chi^{(k,1)}[\tilde{X}_k]\right\rangle_{[Z_k]} = \epsilon_{z_k}\left[Z_k\right] ,$$

where z_k is the value of the k-th carry bit. The k-th digit of the sum y_k can further be obtained by the contraction

$$\left\langle \beta^{\mp,k}\left[Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}\right], \chi^{(k-1,2)}[Z_{k-1}], \chi^{(k,0)}[X_k], \chi^{(k,1)}[\tilde{X}_k]\right\rangle_{[Y_k]} = \epsilon_{y_k}\left[Y_k\right] .$$

Note that the hypergraph representing this instance is a tree and by Thm. **??** also the message passing scheme of Algorithm **??** is guaranteed to produce the exact contractions. We can exploit this fact for example in the efficient computation of averages of the summation digits, when we have an elementary distribution of input digits. We emphasize that the directed belief propagation Algorithm **??** is exact even if the hypergraph fails to be a tree, provided that we have directed and boolean tensors..

Chapter 6
Causality

Abstract We here describe how to represent Causal AI models in the tnreason formalism.

We find tensor network models for the different rungs of the *ladder of causation* [**?**]. The notes are organized to follow the three rungs of the ladder:

- **Rung 1: Association** – captured by Bayesian Networks
- **Rung 2: Intervention** – captured by do-calculus, for which we introduce intervention variables
- **Rung 3: Counterfactuals** – captured by functional causal models (see Chapter 21 in [**?**]) involving response variables

Following the formalism of Chapter 21 in [**?**]:

- Mutilation of the Bayesian network is captured by contractions with one-hot encodings to do-variables D_k.
- Intervention queries are formalized by contractions with one-hot encodings to all do-variables
- Response variables are formalized by selection variables
- Counterfactual queries are modeled as intervention queries on counterfactual twin networks (see [**?**, Chapter 21])

6.1 Rung 1: Association

See main report: Bayesian Networks are tensor networks and probabilistic queries can be answered by their contractions.

6.2 Rung 2: Intervention

So far, there are several hypergraphs to model a probability distributions by a Bayesian Network. We now want to understand the directed hyperedges as causal relationships between the variables, beyond encoding of conditional independencies (see Example 6.1). To this end, we model the effects of interventions, which will distinguish between multiple Bayesian Networks modeling the same joint distribution.

Example 6.1 (Patient treatment outcome) We model a relationship about the treatment of a sick patient and the outcome of the treatment. Whether a treatment is conducted is modelled by the binary variable X_T and whether the patient improves after the treatment by the variable X_0.

There are in general two possibilities of representing the joint distribution of X_T and X_O by a Bayesian Network:

- Treatment directs to outcome:

$$\mathcal{G} = (\{X_T, X_O\}, \{(\emptyset, X_T), (\{X_T\}, \{X_O\})\})$$

- Outcome directs to treatment:

$$\mathcal{G} = (\{X_T, X_O\}, \{(\emptyset, X_T), (\{X_O\}, \{X_T\})\})$$

In the typical intuition, we expect the treatment to have a causal effect on the outcome and not the other way round. From a Bayesian Network perspective, both models are however equivalent and represent the same joint distribution.

Interventions on Bayesian Networks are modeled by the do-calculus [**?**], which we capture here based on do-variables.

6.2.1 Do calculus

We introduce the do-variables D_k taking values in $[m_k + 1]$ for each variable X_k of dimension m_k, which model the effects of interventions. The states are interpreted as

- $d_k \in [m_k]$: An intervention setting X_k to the value d_k
- $d_k = m_k$: No intervention on X_k

Definition 6.1 The causally augmented conditional probability core is defined as

$$\begin{aligned}\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] =& \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] \otimes \epsilon_{m_k}\left[D_k\right] \\ &+ \sum_{d_k \in [m_k]} \epsilon_{d_k}\left[X_k\right] \otimes \epsilon_{d_k}\left[D_k\right] \otimes \mathbb{I}\left[X_{\mathrm{Pa}(k)}\right] .\end{aligned}$$

The causally augmented Bayesian Network is the tensor network

$$\left\langle \{\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d]\} \right\rangle_{\left[X_{[d]}, D_{[d]}\right]} .$$

The Bayesian Networks investigated so far are retrieved by contractions with $\epsilon_{m_k}[D_k]$. This corresponds with the situation, where no interventions are performed on any variable.

Lemma 6.1 *For any Bayesian Network* $\mathbb{P}\left[X_{[d]}\right]$ *on a directed acyclic hypergraph* $\mathcal{G} = ([d], \mathcal{E})$ *we have*

$$\mathbb{P}\left[X_{[d]}\right] = \left\langle \{\mathbb{P}^c\left[X_k \middle| D_k, X_{\mathrm{Pa}(k)}\right] \,:\, k \in [d]\} \cup \{\epsilon_{m_k}[D_k] \,:\, k \in [d]\}\right\rangle_{\left[X_{[d]}\right]} .$$

Proof. By construction we have

$$\mathbb{P}^c\left[X_k \middle| D_k = m_k, X_{\mathrm{Pa}(k)}\right] = \mathbb{P}\left[X_k \middle| X_{\mathrm{Pa}(k)}\right] .$$

Performing this on each causally augmented conditional distribution retrieves thus the original Bayesian Network. □

6.2.2 Intervention Queries

In a Bayesian Network, when intervening on a variable X_v, the corresponding conditional probability gets trivialized. The probability tensor is then captured by

$$\mathbb{P}\left[X_{\mathcal{U}} \middle| \mathrm{do}\left(X_{\mathcal{W}} = x_{\mathcal{W}}\right)\right] = \left\langle \{\mathbb{P}\left[X_v \middle| X_{\mathrm{Pa}(v)}\right] \,:\, v \notin \mathcal{W}\} \cup \{\frac{1}{m_v}\mathbb{I}\left[X_v, X_{\mathrm{Pa}(v)}\right] \,:\, v \in \mathcal{W}\} \cup \{\epsilon_{x_v}[X_v] \,:\, v \in \mathcal{W}\}\right\rangle_{\left[X_{\mathcal{U}}\right]} .$$

Since $\frac{1}{m_v}\mathbb{I}\left[X_v, X_{\mathrm{Pa}(v)}\right]$ is directed with $X_{\mathrm{Pa}(v)}$ incoming and X_v outgoing, the partition function of the network stays 1 and the normalization is captured by the contraction.

6.2.3 Simplifications

In general, when all variables are observable, intervention queries can be expressed by a collection of conditional queries (since any cpd of a Bayesian network is a conditional query, and we contract all except those with interventions on the head variables). In realistic scenarios, however, not all variables are observable (for example, the smoking gene has been only assumed, but no observations have been made). We are interested in situations, where intervention queries can be answered by conditional queries of observable variables. We can proof them in the tensor network formalism based on network separations, which contribution to contractions are scalar multiplications, which therefore are dropped in normalizations.

An example for a simplification is given by the randomized controlled trial.

Example 6.2 (Randomized controlled trial) The randomized controlled trial is a joint distribution of variables

- Confounding variable X_C summarizing other factors affecting the outcome.

- Treatment assignment X_T with values in $\{0, 1\}$. Here, 1 indicates assignment to the treatment group and 0 to the control group. The experiment is conducted such that the treatment variable is independent of the confounding variable X_C.
- Outcome X_O .

We want to estimate the causal effect of the treatment on the outcome, namely calculate the intervention query

$$\mathbb{P}\left[X_O \middle| D_T = 1\right] .$$

By design of the randomized control, the confounding variable has no causal effect on X_T. Thus, we have

$$\mathbb{P}\left[X_O \middle| D_T = 1\right] = \mathbb{P}\left[X_O \middle| X_T = 1\right]$$

that is, the intervention query is equal to the conditional probability.

6.2.3.1 Backdoor Criterion

More general than in the randomized controlled trial we have to deal with confounding variables. The Backdoor Criterion and the Frontdoor Criterion exploit mediator variables X_M, which are also observable and block certain causal paths to unobserved confounding variables X_U.

Lemma 6.2 (Backdoor Criterion) *When X_M blocks any backdoor path from X_0 to X_1 and no node in X_M is a descendant of X_0, then*

$$\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] = \left\langle \mathbb{P}\left[X_1 \middle| X_0 = x_0, X_M\right], \mathbb{P}\left[X_M\right] \right\rangle_{[X_1]} .$$

Proof. By assumption, we can contract all variables beside $X_{[2]}, X_M$ into cpds, and have a simplified causal model (see Figure 6.1a). We have

$$\mathbb{P}^c\left[X_0 \middle| D_0 = x_0, X_M\right] = \epsilon_{x_0}\left[X_0\right] \otimes \mathbb{I}\left[X_M\right]$$

and thus

$$\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] = \left\langle \mathbb{P}\left[X_1 \middle| X_0 = x_0, X_M\right], \mathbb{P}\left[X_M\right] \right\rangle_{[X_1]} .$$

□

Lemma 6.3 (Backdoor Criterion) *When X_M blocks any front door path from X_0 to X_1 and there is no confounder between X_0 and X_M, as well as between X_M and X_1, then*

$$\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] = \left\langle \mathbb{P}\left[X_M \middle| X_0 = x_0\right], \left\langle \mathbb{P}\left[X_1 \middle| X_M, X_0\right], \mathbb{P}\left[X_0\right] \right\rangle_{[X_1, X_M]} \right\rangle_{[X_1]} .$$

Note that the inner contraction has X_0 as a closed variable.

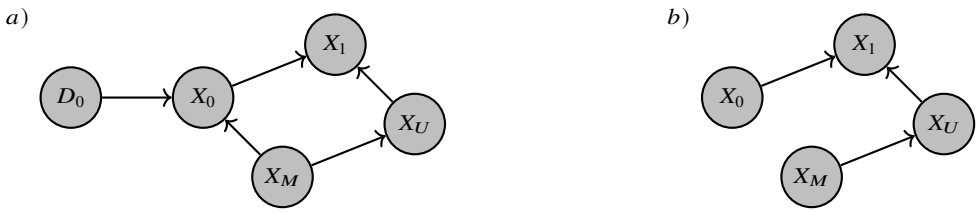


Fig. 6.1 Causal graph for the Backdoor Criterion. a) The mediator variable X_M blocks any backdoor path from X_0 to X_1, potentially through a confounding variable X_U. b) The effect of the intervention on X_0 reduces the Bayesian network effectively to the shown subgraph.

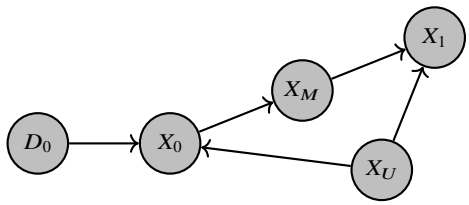


Fig. 6.2 Causal graph for the Frontdoor Criterion.

Proof. We show the claim for the situation in Figure 6.2

$$\begin{aligned}\mathbb{P}^c\left[X_1 \middle| D_0 = x_0\right] &= \left\langle \mathbb{P}\left[X_M \middle| X_0 = x_0\right], \mathbb{P}\left[X_1 \middle| X_M, X_U\right], \mathbb{P}\left[X_U\right]\right\rangle_{[X_1]} \\ &= \left\langle \mathbb{P}\left[X_M \middle| X_0 = x_0\right], \left\langle \mathbb{P}\left[X_1 \middle| X_M, X_U\right], \mathbb{P}\left[X_U\right]\right\rangle_{[X_1, X_M]}\right\rangle_{[X_1]} .\end{aligned}$$

We note that

$$\begin{aligned}\left\langle \mathbb{P}\left[X_1 \middle| X_M, X_U\right], \mathbb{P}\left[X_U\right]\right\rangle_{[X_1, X_M = x_M]} &= \mathbb{P}^c\left[X_1 \middle| D_M = x_M\right] \\ &= \left\langle \mathbb{P}\left[X_1 \middle| X_M, X_0\right], \mathbb{P}\left[X_0\right]\right\rangle_{[X_1, X_M = x_M]}\end{aligned}$$

Here we used in the second equation that the variable X_0 blocks all backdoor paths from X_M to X_1 (i.e. the backdoor criterion). Combining both equations proves the claim. □

6.3 Rung 3: Counterfactuals

Following the formalism of Chapter 21 in [**?**], counterfactual queries are modeled as intervention queries on counterfactual twin networks. Counterfactual twin networks are two copies of the original causal model, one representing the factual world and one representing the counterfactual world. They further share response variables, which cap-

ture the functional relationships between variables in the causal model. These response variables are modeled as selection variables in the tnreason formalism.

Example 6.3 (Patient counterfactual query) Let us assume, that we know whether a patient did not get assigned to the treatment group, and we know whether he did improve. We want to ask, whether he would have improved, if he had been assigned to the treatment group. This is not an intervention query, but a counterfactual query, since we reason about a different outcome, although we already know that the patient has not been treated.

The causally augmented Bayesian Network is not rich enough to answer such queries. Let us consider a situation where X_T is chosen uniformly from $[2]$. There might be situations, where the probabilistic (and intervention) queries are the same, but the counterfactual queries differ:

(i) The outcome is independent of the treatment.
(ii) With probability 0.5, the treatment always leads to improvement and missing the treatment not. And also with probability 0.5, the treatment never leads to improvement, but missing the treatment does.

Intuitively, in the situation (i), the counterfactual query should return probability $1/2$ for improvement under treatment. In the situation (ii), the counterfactual query returns the same value with probability 1 as observed in the real world. Both cases are however indistinguishable by causal models itself, since in both cases the causally augmented conditional probabilities are

$$\mathbb{P}^c\left[X_T \middle| D_T\right] = \frac{1}{2}\mathbb{I}\left[X_T\right] \otimes \epsilon_2\left[D_T\right] + \left(\sum_{d_T \in [2]} \epsilon_{d_T}\left[D_T\right] \otimes \epsilon_{d_T}\left[X_T\right]\right)$$

and

$$\mathbb{P}^c\left[X_O \middle| D_O, X_T\right] = \frac{1}{2}\mathbb{I}\left[X_O, X_T\right] \otimes \epsilon_2\left[D_O\right] + \left(\sum_{d_O \in [2]} \epsilon_{d_O}\left[D_O\right] \otimes \epsilon_{d_O}\left[X_O\right] \otimes \mathbb{I}\left[X_T\right]\right).$$

To answer counterfactual queries, we therefore have to also reason about the mechanisms behind the causal relationships, which we do by introducing response variables in the following.

6.3.1 Response Variables

We introduce response variables L_k, such that X_k has a deterministic dependence on L_k and the parents of a variable to the variable itself. The dependence is captured by a function

Fig. 6.3 Introduction of response variables: a) as a Bayesian network, b) decoration by the basis encoding of the corresponding function.

$$q_k \,:\, [p_k] \times \left(\bigtimes_{\tilde{k} \in \mathrm{Pa}(k)} [m_{\tilde{k}}] \right) \to [m_k] \,.$$

The response augmented conditional probability core is defined as (see Figure 6.3):

$$\mathbb{P}^f \left[X_k \middle| L_k, X_{\mathrm{Pa}(k)} \right] = \beta^{q_k} \left[X_k, L_k, X_{\mathrm{Pa}(k)} \right]$$

6.3.2 Counterfactual Twinned Network

To answer counterfactual queries, we now construct the counterfactual twinned network (see Figure 6.4).

Definition 6.2 Let there be a response augmented Bayesian Network on a directed acyclic hypergraph $\mathcal{G} = ([d], \mathcal{E})$ with response variables $L_{[d]}$. Further another hypergraph $\tilde{\mathcal{G}}$ and a Bayesian Network on $\tilde{\mathcal{G}}$ modelling the joint distribution of the response variables $L_{[d]}$. Be build counterfactual copies $\tilde{X}_k$ of each variable X_k, which are controlled by the same response variables L_k as the factual variables X_k. The counterfactual twinned network is then the tensor network consists of the causal and response augmented conditional probability cores to $X_{[d]}$, the conditional probability cores of the response variables $L_{[d]}$ and the response augmented conditional probability cores to the counterfactual variables $\tilde{X}_{[d]}$.

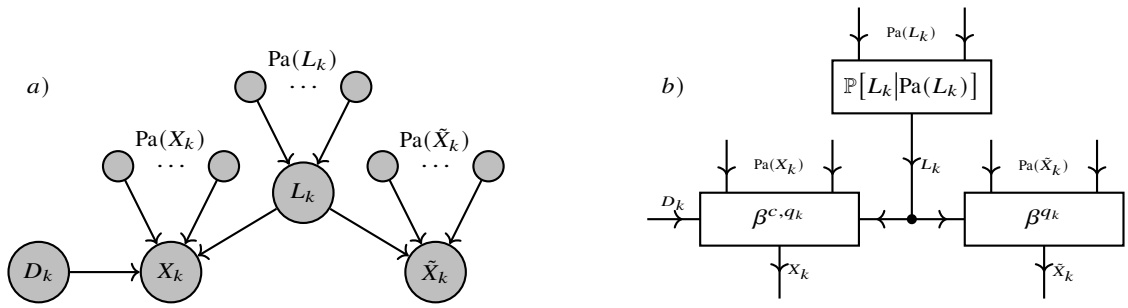


Fig. 6.4 Counterfactual twinned network: a) as a Bayesian network. b) Decorations by causal augmented basis encodings by a do-variable for X_k, a conditional probability distribution for L_k and a basis encoding for the counterfactual variable $\tilde{X}_k$.

Part II
Computation-Activation Networks

Use the template *part.tex* together with the document class `SNmono` (monograph-type books) or `SNmult` (edited books) to style your part title page and, if desired, a short introductory text (maximum one page) on its verso page.

Chapter 7
Elementary Computation-Activation Networks

Abstract Dicuss here the case of elementary activation, Hard, Markov and Hybrid Logic Networks.

Topics

- Exponential Families (Report I.2.3)

Chapter 8
Mean Polytopes and Maximum Entropy Distributions

Abstract Beyond elementary: Study generic maximum entropy distributions. Motivation from the chapter before

Chapter 9
Variational Inference

Abstract We present here variational inference methods to reason on Computation-Activation Networks.

Topics:

- I.3.4 (Exponential family) Variational inference
- I.3.7 Mean field methods

Chapter 10
Inductive Reasoning

Abstract In this chapter, we investigate the problem of selecting statistics.
Topics:

- II.7 Formula Selecting Networks
- II.9.5 Structure Learning in Hybrid Logic Networks

Chapter 11
Probabilistic Guarantees

Abstract Here we show probabilistic guarantees for the success of the reasoning mechanisms on Computation-Activation Networks.

Topics: Chapter II.10

Chapter 12
Quantum Inference

Abstract Quantum Circuits are naturally tensor networks. This motivates the investigation, which contractions can be efficiently performed on a Quantum Computer.

Chapter 13
Probabilistic Relational Models

Abstract We investigate in this chapter, how Compuation-Activation Networks can be trained given data in the form of a first-order logic world.

Part III
Contraction Calculus

Glossary

Use the template *glossary.tex* together with the Springer Nature document class SVMono (monograph-type books) or SVMult (edited books) to style your glossary in the Springer Nature layout.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

glossary term Write here the description of the glossary term. Write here the description of the glossary term. Write here the description of the glossary term.

Solutions

Problems of Chapter ??

?? The solution is revealed here.

?? Problem Heading
(a) The solution of first part is revealed here.
(b) The solution of second part is revealed here.